



Enhanced data fusion of ultrasonic and stereo vision in real-time obstacle detection

Farshad Gholami¹ · Esmaeel Khanmirza¹ · Mohammad Riahi¹

Received: 25 October 2022 / Accepted: 5 May 2023 / Published online: 30 May 2023
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2023

Abstract

In this research, the accuracy and speed of obstacle detection in data fusion of ultrasonic and stereo vision have been improved. The smoothness assumption has been used in such a way that the responses are significantly improved without increasing calculation. In addition, with the development of the proposed method to run on the graphics card, the cross-checking process has been done without the need to change the reference image and without more calculation of the cost function. The results of this study show that the proposed method improved the quality of the responses compared to the previous study, and the obstacle detection rate in intelligent vehicles has increased to 41 pairs of frames per second. This processing rate is 477.40 times faster than the usual local stereo method and 33.77% faster than the previous study on the data fusion of ultrasonic and stereo vision.

Keywords Data fusion · Ultrasonic · Stereo vision · Intelligent vehicle

1 Introduction

Today, traffic has become one of the most important issues of humanity. One of the solutions to solve traffic is to approach this problem as a collective issue. This approach tries to distribute traffic by controlling the entrances of the streets by using traffic lights and providing smooth traffic. In this regard, Wang et al. have modeled and controlled traffic in intersections [1]. Wang et al. also used a genetic algorithm to provide a solution for scheduling traffic signals, thereby making traffic smoother [2]. In another approach, traffic is considered a problem with intelligent components. This category of research is divided into the following.

- Perception layer
- Decision layer

- Navigation layer
- Control layer

The most basic and essential layer in achieving an intelligent vehicle is the layer of perception. Wang has categorized and reported these sensors well in a review article [3]. This classification has been studied in active, inactive, and hybrid sensors.

1.1 Active sensors

These sensors are one of the most important tools for detecting the environment in intelligent vehicles. The famous types of these sensors are Time-of-flight(TOF) and light detection and ranging(LiDAR). However, the critical issues in using these types of sensors are their prices and the limitation of simultaneously using the same kinds of sensors. Despite the explanations, researchers such as Hong Wong et al. have proposed a method for real-time vehicle detection based on LiDAR sensor data [4]. They created a points cloud from the sensor data and then clustered the data and found car candidates.

✉ Esmaeel Khanmirza
khanmirza@iust.ac.ir
Farshad Gholami
f_gholami@mecheng.iust.ac.ir
Mohammad Riahi
riahi@iust.ac.ir

¹ School of Mechanical Engineering, Iran University of Science and Technology, Narmak, 1684613114 Tehran, Iran

1.2 Passive sensors

One essential passive sensor in smart cars is the vision camera. Cameras are used in a variety of arrangements.

1.2.1 Monocular arrangement

In this type of usage, video cameras are usually used in research accompanied by machine learning [5]. The fact that the output of these methods is highly dependent on the richness of the data used in the training process; makes them unreliable due to environmental conditions. Furthermore, monocular cameras, such as structure from motion (SFM), Focus–Defocus, and optical flow methods, are used in scanning the environment by this camera arrangement. Due to the presumption in using these methods, they have not been used in this study [6, 7].

1.2.2 Multi-camera arrangement

This method takes two or more images simultaneously. In this method, the matching process is performed after calibrating the cameras and correcting the captured images. Through the matching process, pixels corresponding to the points of the environment are found in the rectified images. Due to the efficiency of this type of arrangement, this method has been used in various fields such as vibration [8, 9], deflection-shape identification [10], agricultural harvesting, robotic works, and automatic vehicles [11, 12]. Despite the reliability of this method, it has a high computational load. However, several researchers developed methods based on the usage of multiple cameras. These studies can be categorized into the following categories as Local, Global, learning, and Feature-based matching methods.

The corresponding disparity will be found in the local methods by matching the window and the cost function, which is calculated for all pixels and all disparities. In this regard, researchers have presented a variety of cost functions. Han et al. have proposed a cost function for windows with weight adaptable to sparse aggregators [13]. Researchers such as Qiao et al. have changed the shape of windows [14]. Wu et al. have also proposed windows with adaptive weights to overcome the matching problem in areas with low and high textures [15]. Moreover, Ye et al. tried to improve the performance of their local methods by the initial disparity estimation and the lightweight pyramid matching network in real-time disparity estimation [16].

In global matching methods, in addition to the similarity of the intensity smoothness assumption used in the matching process. In this assumption, it is assumed that the pixels adjacent to each pixel have a smooth difference in depth

[17]. Although it improves the quality of results, it greatly increases the computational load. Therefore, researchers have resorted to methods such as graph cuts to achieve the answer [18, 19]. Researchers such as Lu et al. have also developed graph cut-off methods to optimize global stereo methods [20]. In addition, researchers have modified and changed the global stereo method to reduce the computational load in global methods, known as the semi-global methods [21]. Some researchers, like Toledo et al., used the sequence of stereo images to reduce computation load in semi-global stereo [22]. In the training-based method, key points are found and matched by the learned stereo process [23]. In a review article, Hamid et al. have explained stereo vision methods based on deep learning [24]. Because of the importance of data richness, this method was not used in this study. In feature-based stereo vision methods, the features that exist in the environment have been found as key points [25]. Lee et al., used characteristics of road lines in identifying the environment [26], and Xue et al. have used edge matching to focus on the important features and deal with occlusion boundaries as well as untextured regions [27]. Moreover, Si et al. have used algorithms such as SIFT to find key points [25]. Researchers have also used features such as the red color of the rear light of the cars and symmetry in the stereo matching [28]. Although these methods reduce the points in the matching process, the reliability of these methods can be reduced due to the possibility of contradictory instances. Therefore, this type of method has not been used in this study.

1.3 Data fusion of sensors

By sensor data fusion, it is possible to overcome the defects of the individual uses of sensors. Li et al., by emitting the linear pattern of light to the environment as texture and capturing stereo images, have increased the passive stereo accuracy [29]. Shuang et al. improved Li et al. results by emitting the dot pattern of light on the environment [30]. By Data fusion of sensors, speeding up the detection of the environment can be performed. For example, in a previous study, current researchers used a data fusion of ultrasonic and stereo vision to reduce stereo vision computations and achieved obstacle detection 316 times faster than local stereo vision [31]. As a result, the data fusion of ultrasonic and stereo vision has been used in this study.

1.4 Parallel computing

Many researchers tried developing stereo vision methods to speed up their execution. Therefore, they proposed methods for executing on graphics cards or FPGA [32]. The present study has also suggested implementing the proposed algorithm to run on the graphics cards in parallel

mode, which can be performed cross-checking without changing the reference image [31].

According to the discussion above, in this study, researchers have used the local truncated stereo method with data fusion of ultrasonic data and stereo vision. The main contributions of the current study are as follows:

- The matching process for truncated stereo vision with the data fusion of ultrasonic data was enhanced. It is achieved by considering the limited smoothness assumption in a way that caused the improvement in the quality of the results without adding a significant computation load in the matching process for the proposed truncated matching process.
- Develop an implementation for truncated in the matching process with the proposed matching window to execute in parallel mode.

Researchers have increased the calculation speed by 32.13% compared to the previous study [31] and achieved 41 pairs of frames per second obstacle detection, 465.93 times faster than local stereo vision. The related work is explained in the following, and the proposed method is described. Then, the results of this research are reported in two parts: improvement in accuracy and increasing speed.

2 Related study

The following describes the bases of the truncated stereo vision by data fusion of ultrasonic and stereo vision. The previous research suggested reducing the search range in the matching process by data fusion of ultrasonic data and stereo vision. In Fig. 1, the proposed set is shown.

In the local truncated matching with data fusion of ultrasonic data and stereo vision method, the corresponding pixels within a window with dimensions of

$(2w + 1) \times (2w + 1)$ will be found by calculation of the cost function in Eq. 1:

$$localCost = \sum_{i=-w}^w \sum_{j=-w}^w (I_L(x+i+d, y+j) - I_R(x+i, y+j))^2 \quad (1)$$

where x and y are the column and row of anchor pixel in matching window and d is the disparity and I indicates the intensity of pixels and also sub-index L and R indicates left and right image. In the previous study, the researchers suggested that only three layers could be sufficient instead of calculating the cost function for a large number of disparities. It helped to reduce the computation load. Therefore, by reporting the distance of the obstacle using the ultrasonic sensor, the disparity and the layer that consists of the barrier have been identified. Therefore, the cost function was calculated only for the amount of deviation in Eq. 2:

$$detectedDisparity - 1 \leq d \leq detectedDisparity + 1 \quad (2)$$

By reducing the number of detection layers, the accuracy of responses decreased as same as the computational load. In the previous study, it was suggested that calculating only 5 cost functions and using a Median filter is properly sufficient to provide an acceptable response. The median filter is used after identifying the pixels that are part of obstacles. The median filter with the same matching window size was applied to all the results. Therefore, if more than 50% of neighbors in the filter window indicate the presence of the obstacle, then it will be confirmed that the pixel was a part of the obstacle. With this filter, the noise in the response was removed, and the answer was improved.

Figure 2 shows an overview of the omitted cost function and the five calculated cost functions needed for matching and cross-checking process. It should be noted that according to the parallelization method which was presented before, the number of cost function calculations for each pixel was reduced from 5 to 3.

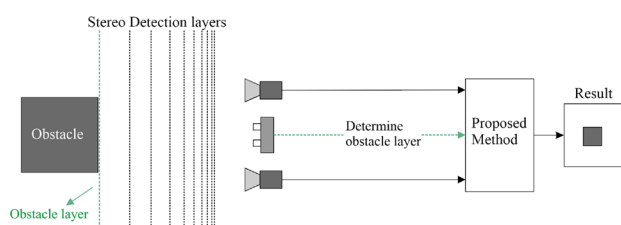


Fig. 1 The proposed setup of ultrasonic and stereo vision in the previous study [31]

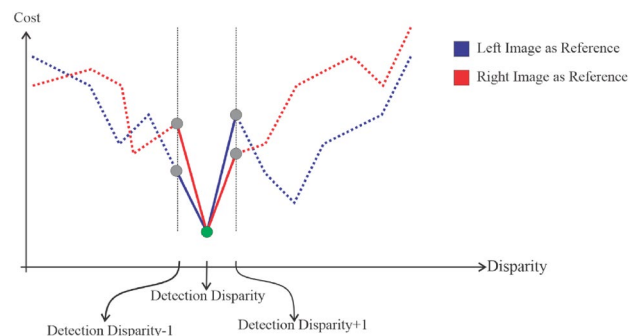


Fig. 2 Truncated cost functions calculation by data fusion of ultrasonic and stereo vision

2.1 Data

To show the effectiveness of the proposed method, the researchers have created a virtual environment in Blender software and produced the data with it. The produced data have been used in previous and current studies to help compare their results. These data are shown in Fig. 3.

3 Algorithm

In the following, the innovations made to the method of truncated stereo vision with ultrasonic data have been explained in two parts: improvement of accuracy and increasing speed.

3.1 Improvement of accuracy

In the stereo vision method, based on the global matching, the depth of each pixel in the view is considered in addition to the similarity of light intensity in the vicinity of the

anchor pixel. Therefore, smoothness assumption, causes the number of cost functions to increase as Eq. 3:

$$\text{num Of Calculation} = (NR \times NC)^{ND} \quad (3)$$

where NR is number of rows in images, NC is number of their columns and ND is number of disparity. By considering the smoothness assumption in only each window, with dimensions $(2w + 1) \times (2w + 1)$ and the number of layers (ND), the number of the cost function to be calculated per pixel will be equal to Eq. 4:

$$\text{num Of Calculation} = ((2w + 1) \times (2w + 1))^{ND} \quad (4)$$

Fig. 4 shows a view of one of the situations that happens in return for the above assumption.

Now, if the calculation is restricted to the same three layers used in the previous method, the number of calculations per pixel will be reduced to the value given in Eq. 5:

$$\text{num Of Calculation} = ((2w + 1) \times (2w + 1))^3 \quad (5)$$

If the smoothness assumption is restricted to only one horizontal or vertical direction, the number of calculations will be equal to Eq. 6. Figure 5 shows a view of this situation.

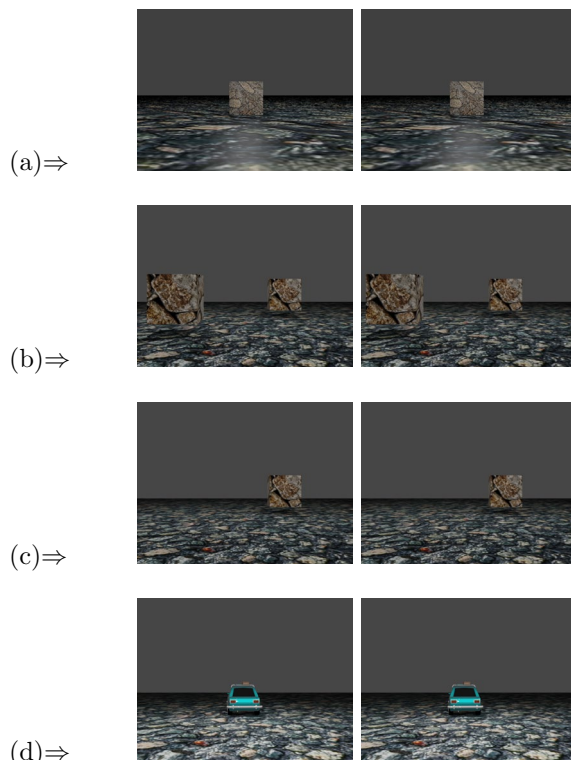


Fig. 3 Each row represents the captured stereo vision in the virtual environment. Left to right indicate the left and right captured images [31]

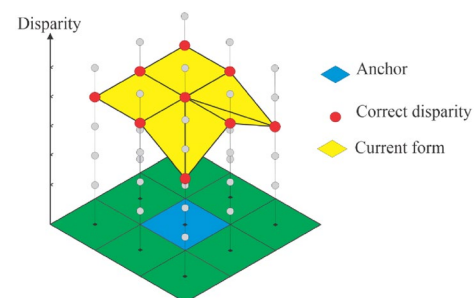


Fig. 4 A possible situation for the depth of pixels in the matching window

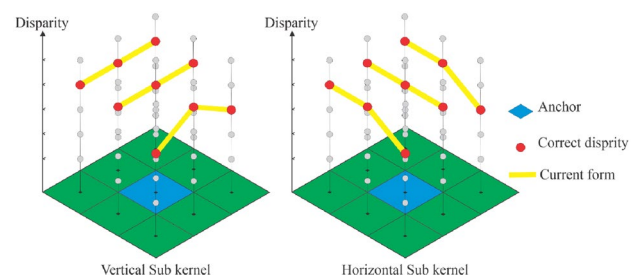


Fig. 5 Restricting the smoothness assumption horizontally or vertically

$$\text{num Of Calculation} = (2w + 1)^3 \times (2w + 1) \quad (6)$$

Now, if the smoothness assumption in vertical or horizontal neighborhoods is limited to only the three states in Fig. 6, the number of calculations will be reduced to Eq. 7:

$$\text{num Of Calculation} = (2w + 1) \times (3 + 1) \quad (7)$$

In the above statement, number 3 corresponds to three layers for which the cost function is calculated, and number 1 in the second parenthesis of Eq. 7 is due to the review of the sub kernels' results. This means that after calculating the cost function using Eq. 8 for the vertical sub-kernel or Eq. 9 for the horizontal sub-kernel, a comparison is made regarding the acceptability of the result, where v sub-index indicates *Vertical* and h sub-index indicates *Horizontal* sub-kernel:

$$\text{Cost}_v(m) = \sum_{j=-w}^w (I_L(x + m - w + d, y + j) - I_R(x + m - w, y + j))^2 \quad (8)$$

$$\text{Cost}_h(n) = \sum_{i=-w}^w (I_L(x + i + d, y + n - w) - I_R(x + i, y + n - w))^2 \quad (9)$$

where n and m are sub-kernel number in vertical and horizontal kernel. The comparison for acceptance is being done after calculating the cost functions for the sub-kernels for three disparities. For each sub-kernel, if the mid layer cost function is less than the previous and last layers, that sub-kernel is considered an acceptable candidate for an obstacle, and if the number of accepted candidates exceeds the

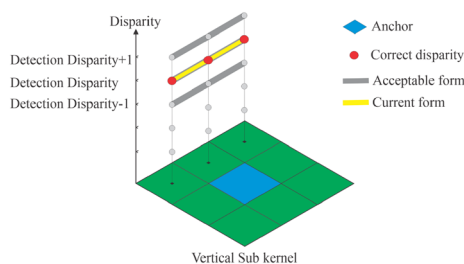


Fig. 6 Limitation of possible situations in the vertical sub-kernel to the three-layer

predefined threshold value, that anchor pixel is identified as a pixel of the obstacle. Although this assumption does not increase the order of calculations compared to the previous study, at the same time the smoothness assumption is applied almost sufficiently. This means that it is possible to consider reduced smoothness without increasing the number of calculations which increase the quality of responses. In the results section, the improvement results are presented numerically and visually. In order to apply the cross-check to improve the results as much as possible, the following pseudocode should be used. It should be noted that according to the cross-checking process in the proposed method, the number of calculations per pixel will be equal to the Eq. 10:

$$\text{num Of Calculation} = (2w + 1) \times (5 + 1) \quad (10)$$

It is important be noted that by rederive the parallel processing method presented in the previous article for the present method, the number of calculations will be reduced to Eq. 7 again. This means that the cross-check process will be possible without any calculation of the new cost function. This is explained in the increasing speed section. According to the explanation, The time complexity of this algorithm (O) is as Eq. 11, which is equal to the previous study, although it helps imply smoothness without increasing calculation. Although the time complexity of this algorithm in both studies is similar, according to differences in the GPU implementation in these studies, the run time of this study is reduced alongside improving the quality of results:

$$\text{Time Complexity} = O((NR \times NC) \times (2w + 1)^2 \times 4) \quad (11)$$

3.2 Increasing speed

One of the most important requirements for using stereo vision in smart vehicles is to execute the algorithm in real-time. Therefore, in addition to increasing the accuracy of obstacle detection, the proposed method must be executed in a very short period of time. On the other hand, due to the high number of pixels and due to the independence of the corresponding calculations, it is possible to execute the proposed algorithm in parallel. In this research, it is also proposed a method for implementing the presented algorithm in parallel on the graphics card. In this regard, each of the left and right images are initially converted into a one-dimensional array. This conversion is in such a way that the rows of each image are arranged in sequence from the top in a one-dimensional array. Therefore, the following equation

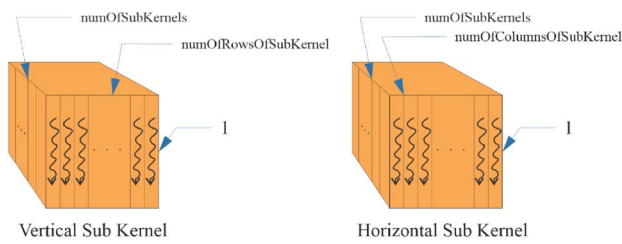


Fig. 7 View of computational blocks for horizontal and vertical sub-kernels

can be used to obtain the light intensity in each image in an equivalent one-dimensional form:

$$I(i, j) = I_{1D}(j \times \text{num Of Columns}_i) \quad (12)$$

where “ i ” corresponds to the position of the pixel in the x -direction and “ j ” corresponds to the position of the pixel in the y -direction in the image of 2D form. The corresponding 1D array of both left and right images are copied to the Global memory on the graphics card. Then the calculations are performed using computational threads. Therefore, a Grid of Blocks has been created in which there are computational threads according to Fig. 7. In this Grid, there are three pages of calculation blocks, each of which is responsible for performing calculations related to a certain amount of disparity. The middle page is responsible for calculations related to the disparity extracted from ultrasonic data and the other two layers are dedicated to performing cost function calculations for the prior and posterior disparity of the detection layer. The numbers of rows and columns on each of these pages are equal to the numbers of anchors’ pixels with matching window size ($\text{Kernel Size} \times \text{Kernel Size}$) to find obstacles in the layer detected by the ultrasonic sensor data (SelectedDisparity). Therefore, if the position of a block in the Grid of blocks is $(\text{block}_x, \text{block}_y, \text{block}_z)$ then their variation are as follows:

$$\begin{aligned} 0 \leq \text{block}_x &\leq \text{Rows Of Image} - \text{KS} - (\text{SD} + 1) \\ 0 \leq \text{block}_y &\leq \text{Columns Of Image} - \text{KS} \\ 0 \leq \text{block}_z &\leq 2 \end{aligned} \quad (13)$$

where SD indicates selected disparity and:

$$\text{KS} = 2w + 1 \quad (14)$$

The size of each block which is used, depending on whether vertical or horizontal sub-kernel, is as shown in Fig. 7.

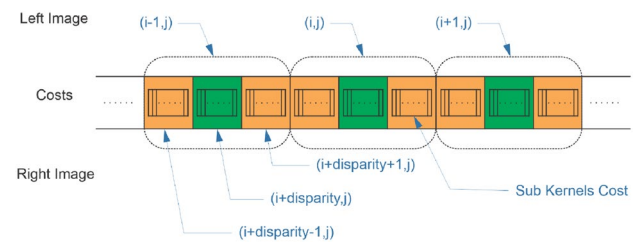


Fig. 8 The form of cost function storage

Therefore, the position of the anchor pixel corresponding to the block with position $(\text{block}_x, \text{block}_y, \text{block}_z)$ is as follows:

$$\begin{aligned} i &= \text{block}_x + w \\ j &= \text{block}_y + w \end{aligned} \quad (15)$$

Also, the amount of disparity for each block is calculated as Eq. 16:

$$\text{disparity} = \text{SD} - 1 + \text{block}_z \quad (16)$$

Then, a thread with position $(\text{thread}_x, \text{thread}_y, \text{thread}_z)$ in the block $(\text{block}_x, \text{block}_y, \text{block}_z)$ is responsible for calculating the square of the difference in light intensity of two pixels to the following positions, if the sub-kernel is horizontal:

$$\begin{aligned} \text{Right Image}_x &= \text{block}_x + \text{thread}_x \\ \text{Right Image}_y &= \text{block}_y + \text{thread}_y \\ \text{Left Image}_x &= \text{Right Image}_x \\ \text{Left Image}_y &= \text{Right Image}_y + \text{SD} - 1 + \text{block}_z \end{aligned} \quad (17)$$

And if the sub-kernel is vertical, these positions will be as follows:

$$\begin{aligned} \text{Right Image}_x &= \text{block}_x + \text{thread}_x \\ \text{Right Image}_y &= \text{block}_y + \text{thread}_y \\ \text{Left Image}_x &= \text{Right Image}_x \\ \text{Left Image}_y &= \text{Right Image}_y + \text{SD} - 1 + \text{block}_z \end{aligned} \quad (18)$$

Now, due to the aforementioned positions and using Eq. 12, the light intensities which are required for each thread is obtained and their difference squares are calculated. After performing the calculations in each block, a thread with coordination $(0, 0, 0)$ calculates the sum of the squares of the sub kernels and generates a vector of “NumOfSubKernel” length, which it places in the

vector whose maintenance cost functions. This vector size is $NumOfPixels \times 3 \times KernelSize$. The storage form will be as shown in Fig. 8.

After completing the calculations of all the blocks, in each block of the middle page, the thread (0,0,0) is responsible for inferring the result. Finally, it puts the result of the inference in one-dimensional arrays with size “numOfPixels”. Finally, this part of the array is delivered as a result. Finally, by converting it to a 2D image, the result is produced.

For inference, each of the green sub-arrays is compared with the equivalent sub-arrays in the two previous and subsequent orange sub-arrays, and if the value of the cost function under the green array is less than the other two, the number of valid candidates is added. And if the number of accepted candidates exceeds the threshold, that pixel will be considered an obstacle. The explanation given was in the case that no cross-checks were done. Now, if the cross-check is activated, in the inference step, each green subarray must be compared with the four orange sub-arrays in its neighborhood, and there is no need to calculate another cost function. Therefore, it does not require any cost function calculation in CPU or GPU, and it will be performed at this stage, and the final answer will be generated. After receiving the results from the GPU, the horizon line in the CPU is removed and results are generated. The corresponding pseudocode for this algorithm to execute in parallel on the GPU is as Pseudocode 1 and 2.

Algorithm 1 Execution of enhanced Cut-SSD with cross check on GPU.

```

1: leftImg, rightImg  $\leftarrow$  ReadStereoImages()
2: selectedDis  $\leftarrow$ 
   getDataUltrasonicSensorData()
3: leftImg1D  $\leftarrow$  convert2DTo1D(leftImg)
4: rightImg1D  $\leftarrow$  convert2DTo1D(rightImg)
5: copyImageToGPU(leftImg1D, rightImg1D)
6: result1DGPU  $\leftarrow$ 
   matching(MaxDisparity, nC, SD, KernelType)
    $\triangleright$  run on GPU
7: result1DInCPU  $\leftarrow$ 
   copyresultFromGPU(result1DGPU)
8: result2DInCPU  $\leftarrow$ 
   convert2DTo1D(result1DInCPU)
9: return result2DInCPU

```

Algorithm 2 Matching method with vertical SubKernel for run on GPU.

Require: *lImage, rImage, numOfColumns, SD*
 $\triangleright$ Dis means Disparity $\triangleright$ left and right images are in 1D format.

Ensure: *Costs* $\triangleright$ 1D array of the calculated cost function for all pixels

```

1: function MATCHING(nC, selectedDis)
2:   rIndexU  $\leftarrow$  blockIdx.x + threadIdx.x
3:   rIndexV  $\leftarrow$  blockIdx.y + threadIdx.y
4:   lIndexU  $\leftarrow$  rIndexU + blockIdx.z - 1 + SD
5:   lIndexV  $\leftarrow$  rIndexV
6:   rIndex  $\leftarrow$  rIndexV  $\times$  numOfColumns + rIndexU
7:   lIndex  $\leftarrow$  lIndexV  $\times$  numOfColumns + lIndexU
8:   index  $\leftarrow$  threadIdx.x + threadIdx.y  $\times$  KS
9:   diff[index]  $\leftarrow$ 
     pow(lImage[lIndex] - rImage[rIndex], 2)
10:  __syncthreads()
11:  if blockIdx.x == 0 AND blockIdx.y == 0
     then
12:    costs  $\leftarrow$  0
13:    for i  $\leftarrow$  0 to blockDim.x do
14:      costs  $\leftarrow$ 
        costs + diff[i + threadIdx.z  $\times$  kernelSize]
15:    end for
16:    allCostsIndex  $\leftarrow$ 
      (blockIdx.y + int(KS/2))  $\times$  nC + (blockIdx.x + int(KS/2))
17:    allCostsIndex  $\leftarrow$ 
      allCostsIndex  $\times$  3  $\times$  KS + blockIdx.z  $\times$  KS + threadIdx.z
18:    allCosts[allCostsIndex]  $\leftarrow$  costs
19:  end if
20:  __syncthreads()
21:  if blockIdx.x == 0 AND blockIdx.y == 0
     then
22:    NASK  $\leftarrow$  countAcceptedSubKernel()
23:    if NASK > AcceptanceThreshold
       then
24:      result[resultIndex]  $\leftarrow$  255
25:    end if
26:    __syncthreads()
27:  end if
28:  return result
29: end function

```

4 Results

To evaluate the efficiency of the proposed method and the improvement given to the previous research [31], the same data presented in that research have been used. Those data are shown in Fig. 3. Two references [31, 33] are selected to compare the results and show the competitive developments. The reference [31] is the closest and most recent state of the art in the fusion of ultrasonic data with stereo vision which is the basis of the current work, and the reference [33] is chosen to compare the improvement in the quality of local stereo vision results alongside the speed-up of local stereo vision.

As described in the description of the proposed method in the algorithm section, in this study, to imply smoothness

implicitly, each kernel is converted to a number of vertical or horizontal sub-kernels, and each kernel with a threshold limit is the condition for accepting anchor pixels as an obstacle. In this regard, the obstacle has been detected for the data presented in Fig. 9 in the division of sub-kernel type and the threshold value. These results are presented in five columns. The first and third columns correspond to the vertical sub-kernel and the second and fourth columns correspond to the horizontal sub kernel. In the first two columns, the threshold is 40% and in the second two columns, the threshold of acceptance is 60%. In order to compare the quality of the proposed method, the results of the previous study, are presented in fifth column.

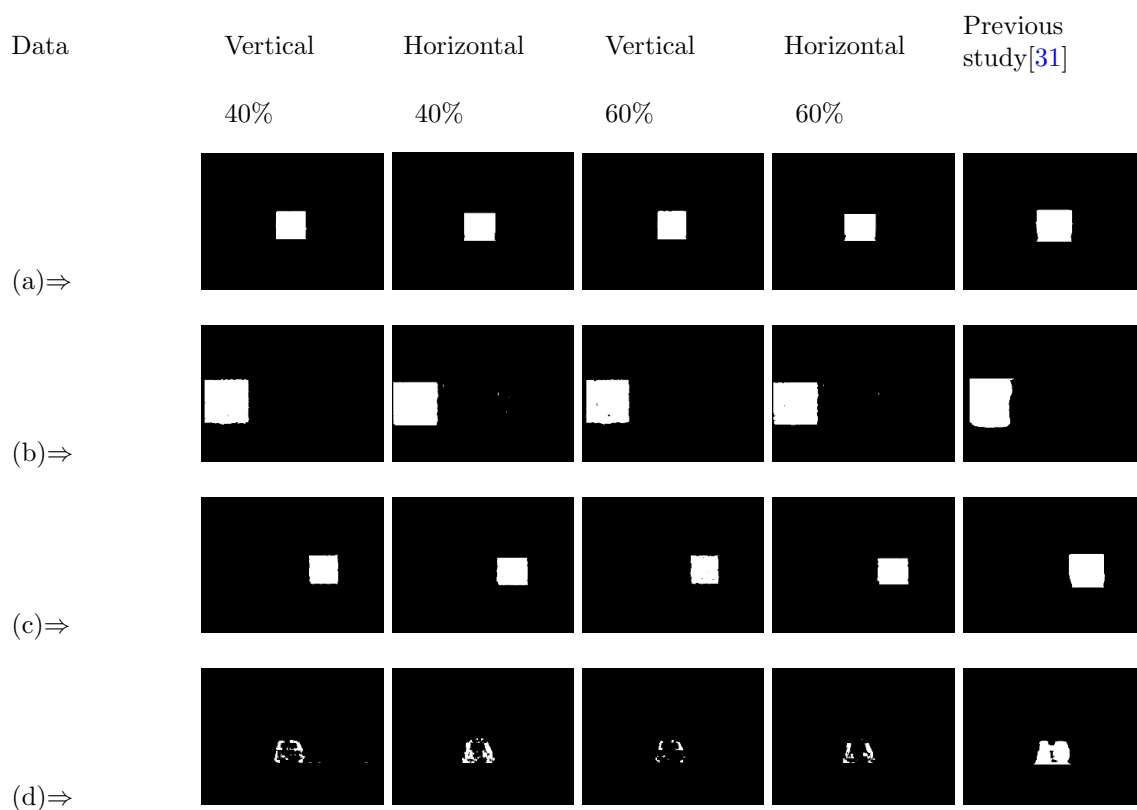


Fig. 9 Comparison of results in different situations. Each row of results indicates the results for the data that are reported in Fig. 3

Table 1 The percentage error in the results

Data	V 40%	H 40%	V 60%	H 60%	[31]	[33]
a	0.87	0.76	0.85	0.81	2.12	4.19
b	1.09	0.78	1.02	0.56	1.07	2.05
c	0.63	0.50	0.58	0.46	2.10	2.33
d	1.75	1.48	1.69	1.85	1.86	2.13

Each row of the first column indicates the series of data that are reported in Fig. 3

Table 2 Average time of three executions of the proposed method for each data series on three hardware configurations on millisecond unit

Data	Configuration 1				Configuration 2				Configuration 3			
	a	b	c	d	a	b	c	d	a	b	c	d
Read images	8.68	8.67	8.70	9.23	8.74	8.82	8.75	8.78	8.90	8.96	8.79	8.76
2D images to 1D	0.93	0.93	0.93	0.93	1.38	1.37	1.39	1.51	1.46	1.42	1.56	1.41
Copy to GPU	0.25	0.22	0.21	0.20	0.80	0.84	0.79	0.67	0.86	0.83	0.81	0.93
Matching	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.03	0.04	0.04	0.03
Copy from GPU	12.49	12.50	12.49	12.85	14.54	14.34	14.81	14.45	8.92	8.86	8.87	8.98
1D result to 2D	0.15	0.14	0.16	0.15	0.24	0.23	0.23	0.23	0.25	0.23	0.23	0.24
Remove horizon line	1.14	1.14	1.14	1.15	1.14	1.14	1.14	1.14	1.14	1.15	1.14	1.14
Total duration	23.65	23.63	23.64	24.53	26.86	26.77	27.14	26.81	21.55	21.50	21.44	21.49
Average	23.86				26.90				21.50			

Where a, b, c, and d indicate the data series used and reported in Fig. 3

Table 3 Comparison between the current study and [31] in terms of the total run time and duration of obstacle detection without considering the duration of copying the results from the GPU

Data	Current study		Study [31]		Improvement	
	TT (ms)	TWC (ms)	TT (ms)	TWC (ms)	TT %	TWC %
a	23.65	11.17	36.03	20.90	34.35	46.57
b	23.63	11.12	35.60	20.42	33.63	45.51
c	23.64	11.15	36.08	20.96	34.47	46.79
d	24.53	11.67	36.41	21.28	32.64	45.15
Avg	23.86	11.28	36.03	20.89	33.77	46.00

As it can be visible, the response quality has improved in comparison to the previous study. Furthermore, Table 1 is presenting a comparison between the percentages of error in the proposed method and the same work that has been done in the previous study. The percentages of errors are calculated with Eq. 19:

$$\text{Percentage Of Error} = \frac{NIPD}{NTP} \times 100 \quad (19)$$

where *NIPD* indicates the number of pixels with incorrect detection, and *NTP* indicates the total number of pixels in the reference image.

As can be seen from Table 1 and the images in Fig. 9, the accuracy of the results has improved despite the elimination of the Median filter process.

4.1 Run time

One of the main contributions of this study is the execution time on GPU. Therefore, three hardware configurations are used to remove hardware ambiguity. These configurations are as follows.

- Configuration 1

- CPU: Intel Core i7-7800k
- GPU: GeForce GTX 1080 Ti

- Configuration 2

- CPU: AMD Ryzen 7 3800X 8-Core
- GPU: Nvidia GeForce RTX 2070

- Configuration 3

- CPU: AMD Ryzen 7 3800X 8-Core
- GPU: NVIDIA GeForce RTX 3070 Ti

Table 4 An average run time comparison between GPU and CPU in millisecond units using hardware configuration 1

Data	Current study	Local [33]	Improvement ratio
a	23.65	11520.45	487.04
b	23.63	11136.37	471.35
c	23.64	11484.33	485.72
d	24.53	11426.91	465.91
Avg	23.86	11392.02	477.40

Each row of the first column indicates the data series reported in Fig. 3

Table 5 A comparison between the CPU run time of the proposed method, [31], and [33]

Data	V 40%	H 40%	V 60%	H 60%	[31]	[33]
a	1038	982	992	1036	1153	11520
b	1014	990	995	989	1153	11136
c	976	983	1012	985	1160	11484
d	979	992	1066	1011	1154	11426

The results are reported on millisecond units. Each row of the first column indicates the data series written in Fig. 3

In Table 2, the execution times in these configurations are reported. Each column of results reported in Table 2 is the average of three execution in each hardware configuration for each data series in milliseconds unit. The average execution time of these configurations shows less deviation from the execution time of Hardware Configuration 1.

The first configuration is equivalent to the system used in the previous work. Therefore, to compare and show the improvement in execution time according to the previous study [31], this configuration is selected, and the execution comparison of this hardware configuration is reported in Table 3. As shown in Table 3, the average total execution time improved by 33.77%, and the execution time without considering the runtime of copying data to GPU is 46.00%. In Table 3, the “TT” indicates the total time, and “TWC” means the total time without copying the result time.

To provide a comparison between the GPU execution of the proposed method and the local stereo vision method, Table 4 is provided. Table 4 provides the average of three execution of the proposed method in GPU and local stereo vision method in CPU by hardware configuration 1. As shown in Table 4, the proposed method is 477.40 times faster than the local stereo vision method [33].

Furthermore, Table 5 provides the comparison between the proposed method and research [31, 33] on the CPU by hardware configuration 1. As shown, the proposed method execution runtime on the CPU has slightly decreased compared to the previous method. However, due to the upgrade proposed in the implementation on GPU, the execution time has been reduced from 36.037 ms to 23.86 ms. This indicates a significant improvement in obstacle detection time alongside increases in the accuracy of results.

5 Conclusion

In this study, researchers increased the accuracy and speed of the truncated stereo vision by data fusion of ultrasonic and stereo vision. By proposing the reduced smoothness assumption, the accuracy of the responses is increased. Researchers have added a limited smoothness assumption, so the computational load does not change and it is also caused to improve the quality of the response. This upgrade has been done in

such a way that after the matching process, there is no need to use a filter to improve the response. In order to reduce the execution time, parallel algorithm presented in the previous study has been adapted for the currently proposed method. In this implementation, the matching process is performed without transferring data back to the GPU and the cross-check process has been performed at once. Furthermore, by rederiving the previous cross-check method for the present study, cross-check has been performed without any cost function calculation. This has led to an improvement in obstacle detection by 33.77% in addition to improvements in response quality. Therefore, if a vehicle needs to scan the environment for each meter of movement, it can reach the speed of 150.88 km/h.

Author contributions FG: wrote the main manuscript text and code Dr. EK: supervisor and contributor Dr. MR: supervisor and contributor.

Declarations

Conflict of interest The authors declare no competing interests.

References

1. Wang, H., Patil, S.V., Aziz, H.A., Young, S.: Modeling and control using stochastic distribution control theory for intersection traffic flow. *IEEE Transact. Intell. Transport. Syst.* **23**, 1885–1898 (2020)
2. Wang, H., Hu, P., Wang, H.: A genetic timing scheduling model for urban traffic signal control. *Inf. Sci.* **576**, 475–483 (2021)
3. Wang, Z.: Review of real-time three-dimensional shape measurement techniques. *Measurement* **156**, 107624 (2020)
4. Wang, H., Zhang, X.: Real-time vehicle detection and tracking using 3d lidar. *Asian J. Control.* **24**, 1459–1469 (2021)
5. Ye, D.H., Li, J., Chen, Q., Wachs, J., Bouman, C.: Deep learning for moving object detection and tracking from a single camera in unmanned aerial vehicles (uavs). *Electronic Imaging* **2018**(10), 466–1 (2018)
6. Fan, L., Zhang, T., Du, W.: Optical-flow-based framework to boost video object detection performance with object enhancement. *Expert Syst. Appl.* **170**, 114544 (2021)
7. Lu, S., Luo, Z., Gao, F., Liu, M., Chang, K., Piao, C.: A fast and robust lane detection method based on semantic segmentation and optical flow estimation. *Sensors* **21**(2), 400 (2021)
8. Yu, L., Pan, B.: Single-camera high-speed stereo-digital image correlation for full-field vibration measurement. *Mech. Syst. Signal Process.* **94**, 374–383 (2017). <https://doi.org/10.1016/j.ymssp.2017.03.008>

9. Durand-Texte, T., Simonetto, E., Durand, S., Melon, M., Moulet, M.-H.: Vibration measurement using a pseudo-stereo system, target tracking and vision methods. *Mech. Syst. Signal Process.* **118**, 30–40 (2019). <https://doi.org/10.1016/j.ymssp.2018.08.049>
10. Gorjup, D., Slavič, J., Babnik, A., Boltežar, M.: Still-camera multiview spectral optical flow imaging for 3d operating-deflection-shape identification. *Mech. Syst. Signal Process.* **152**, 107456 (2021). <https://doi.org/10.1016/j.ymssp.2020.107456>
11. Kim, W.-S., Lee, D.-H., Kim, Y.-J., Kim, T., Lee, W.-S., Choi, C.-H.: Stereo-vision-based crop height estimation for agricultural robots. *Comput. Electron. Agric.* **181**, 105937 (2021)
12. Wu, B., Wang, L., Liu, X., Wang, L., Xu, K.: Closed-loop pose control and automated suturing of continuum surgical manipulators with customized wrist markers under stereo vision. *IEEE Robot. Automat. Lett.* **6**(4), 7137–7144 (2021)
13. Han, C., Liu, W., Jin, L., Jiang, S., Li, H.: Adaptive weight based sparse block aggregation algorithm for stereo matching. In: 2019 9th International Conference on Information Science and Technology (ICIST), pp. 408–412 (2019). IEEE
14. Qiao, W., Xu, Y., Zhang, C., Xu, Z., Huang, J., Xie, P., Lu, J.: An improved adaptive window stereo matching algorithm. *J. Phys.* **1634**, 012066 (2020). (IOP Publishing)
15. Wu, W., Zhu, H., Yu, S., Shi, J.: Stereo matching with fusing adaptive support weights. *IEEE Access* **7**, 61960–61974 (2019)
16. Ye, X., Yan, B., Liu, B., Wang, H., Qi, S., Chen, D., Wang, P., Wang, K., Sang, X.: Improved real-time three-dimensional stereo matching with local consistency. *Image Vis. Comput.* **124**, 104509 (2022). <https://doi.org/10.1016/j.imavis.2022.104509>
17. Woodford, O., Torr, P., Reid, I., Fitzgibbon, A.: Global stereo reconstruction under second-order smoothness priors. *IEEE Trans. Pattern Anal. Mach. Intell.* **31**(12), 2115–2128 (2009)
18. Bleyer, M., Gelautz, M.: Graph-cut-based stereo matching using image segmentation with symmetrical treatment of occlusions. *Signal Process.* **22**(2), 127–143 (2007)
19. Wang, H.-q., Wu, M., Zhang, Y.-b., Zhang, L.: Effective stereo matching using reliable points based graph cut. In: 2013 Visual Communications and Image Processing (VCIP), pp. 1–6 (2013). IEEE
20. Lu, B., Sun, L., Yu, L., Dong, X.: An improved graph cut algorithm in stereo matching. *Displays* **69**, 102052 (2021)
21. Hirschmuller, H.: Stereo vision in structured environments by consistent semi-global matching. In: 2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06), vol. 2, pp. 2386–2393 (2006). IEEE
22. Toledo, J., Lauer, M., Stiller, C.: Real-time stereo semi-global matching for video processing using previous incremental information. *J. Real-Time Image Proc.* **19**(1), 205–216 (2022)
23. Zbontar, J., LeCun, Y.: Computing the stereo matching cost with a convolutional neural network. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1592–1599 (2015)
24. Hamid, M.S., Abd Manap, N., Hamzah, R.A., Kadmin, A.F.: Stereo matching algorithm based on deep learning: a survey. *J. King Saud Univ-Comput. Inform. Sci.* **34**, 1663–1673 (2020)
25. Premaratne, P., Safaei, F.: Feature based stereo correspondence using moment invariant. In: 2008 4th International Conference on Information and Automation for Sustainability, pp. 104–108 (2008). IEEE
26. Lee, C., Lim, Y.-C., Kwon, S., Lee, J.: Stereo vision-based vehicle detection using a road feature and disparity histogram. *Opt. Eng.* **50**(2), 027004 (2011)
27. Xue, T., Owens, A., Scharstein, D., Goesele, M., Szeliski, R.: Multi-frame stereo matching with edges, planes, and superpixels. *Image Vis. Comput.* **91**, 103771 (2019). <https://doi.org/10.1016/j.imavis.2019.05.006>
28. Bertozzi, M., Broggi, A., Fascioli, A., Nichele, S.: Stereo vision-based vehicle detection. In: Proceedings of the IEEE Intelligent Vehicles Symposium 2000 (Cat. No. 00TH8511), pp. 39–44 (2000). IEEE
29. Li, Y., Wang, Z.: Rgb line pattern-based stereo vision matching for single-shot 3-d measurement. *IEEE Trans. Instrum. Meas.* **70**, 1–13 (2020)
30. Shuang, Y., Wang, Z.: Active stereo vision three-dimensional reconstruction by rgb dot pattern projection and ray intersection. *Measurement* **167**, 108195 (2021)
31. Gholami, F., Khanmirza, E., Riahi, M.: Real-time obstacle detection by stereo vision and ultrasonic data fusion. *Measurement* **190**, 110718 (2022)
32. Li, J., Wu, J., You, Y., Jeon, G.: Parallel binocular stereo-vision-based gpu accelerated pedestrian detection and distance computation. *J. Real-Time Image Proc.* **17**(3), 447–457 (2020)
33. Yang, S., Scherer, S.A., Yi, X., Zell, A.: Multi-camera visual slam for autonomous navigation of micro aerial vehicles. *Robot. Auton. Syst.* **93**, 116–134 (2017)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.